PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL
MATERIAL DE APOYO
CREACIÓN DE APIS EN PYTHON

Objetivo: Construir una API REST utilizando Flask y Flask-SQLAlchemy. La API deberá permitir el consumo desde una aplicación web desarrollada con React y Vite, mediante solicitudes HTTP y el intercambio de datos en formato JSON.

¿Qué es un ORM?

Un ORM (***Object-Relational Mapper***) es un toolkit que nos ayuda a trabajar con las tablas de la base de datos como si fueran objetos, de manera que cada tabla se mapea con una clase y cada columna con un campo de dicha clase. Además, también nos permite mapear las relaciones entre tablas como relaciones entre objetos.

Una de las características que más atractivo hacen a los **ORMs** es que se puede cambiar la base de datos sin apenas modificar código. De manera que puede programar la aplicación con un ORM sin pensar que la base de datos es, por ejemplo, Mysql o PostgreSQL.

Flask-SqlAkchemy

Flask-SQLAlchemy es una extensión de ***Flask*** que agrega soporte para el popular mapeador relacional de objetos (ORM) Python (ORM) SQLAlchemy a las aplicaciones de Flask. Su objetivo es simplificar SQLAlchemy con Flask proporcionando algunas implementaciones predeterminadas para tareas comunes.

Para iniciar debemos primero instalar la extensión así:

```
pip install flask_sqlalchemy
```

El siguiente paso es en el archivo donde se crea la aplicación de tipo **Flask**, se debe integrar con **Flask-SqlAlchemy**. Por lo anterior debemos importar a flask-SqlAlkchemy

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy
```

Configurar a la aplicación la cadena de conexión cumpliendo con la siguiente estructura:
(Tomada de: https://flask-sqlalchemy.palletsprojects.com/en/2.x/config/#configuration-keys)

`SQLALCHEMY_DATABASE_URI`	• `sqlite:////tmp/test.db` • `mysql://username:password@server/db`

```
dialect+driver://username:password@host:port/database
```

```
#cadena de conexión a la base de datos
app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql+pymysql://root@localhost/productospython'
```

Después crear el objeto que nos representa la base de datos, asociado a la aplicación flask creada.

```
db = SQLAlchemy(app)
```

Creación de los modelos

Antés de iniciar tener en cuenta lo siguiente:
La clase base para todos sus modelos se llama ***db.Model***. Se almacena en la instancia de ***SQLAlchemy*** que debe crear.

Algunas partes que se requieren en **SQLAlchemy** son opcionales en **Flask-SQLAlchemy**. Por ejemplo, el nombre de la tabla se establece automáticamente a menos que se anule. Se deriva del nombre de la clase convertido a minúsculas y con "CamelCase" convertido a "camel_case". Para anular el nombre de la tabla, establezca el atributo de clase **__tablename__.**

Ejemplo código de la clase **Categoria** de una tabla llamada categorías

```
modelo > categoria.py > Categoria > __repr__
1   from app import db
2
3   class Categoria(db.Model):
4       __tablename__ = 'categorias' #nombre de la tabla
5       idCategoria = db.Column(db.Integer, primary_key=True,autoincrement=True)
6       catNombre = db.Column(db.String(50),unique=True, nullable=False)
7
8       def __repr__(self):
9           return f'{self.catNombre}'
```

En la línea 1 estamos importando el objeto **db** creado en el archivo donde se crea la aplicación. Al definir la clase estamos heredando de la clase Padre ***db.Model*** que representa la clase base para todos los modelos que vayamos a crear en nuestro proyecto. En la línea 4 estamos configurando el atributo **__tablename__** como **categorías** nombre real de la tabla en la base de datos. En las líneas 5 y 6 estamos creando unos atributos a partir de la clase base. En la siguiente imagen se muestra los diferentes tipos más comunes que se pueden utilizar a la hora de crear una columna en el modelo.

Integer	an integer
String(size)	a string with a maximum length (optional in some databases, e.g. PostgreSQL)
Text	some longer unicode text
DateTime	date and time expressed as Python **datetime** object.
Float	stores floating point values
Boolean	stores a boolean value
PickleType	stores a pickled Python object
LargeBinary	stores large arbitrary binary data

La función ***__repr__*** nos permite representar el objeto en formato string.

Situación problémica

Una tienda necesita una solución tecnológica para administrar su catálogo de productos. Actualmente, requiere una interfaz web que permita registrar nuevos productos, consultar el inventario, modificar precios y categorías, y eliminar registros.

Para resolver esta necesidad, se desarrollará una **API REST** en **Flask** conectada a **MySQL**. Posteriormente, se consumirá desde una aplicación React + Vite, separando la lógica de acceso a datos, el procesamiento de solicitudes y las rutas de la API.

Conceptos fundamentales

- **API REST:** interfaz que expone recursos mediante rutas HTTP y permite intercambiar información, generalmente en JSON.
- **Flask:** framework de Python para construir aplicaciones web y servicios API.
- **Flask-SQLAlchemy:** extensión que integra SQLAlchemy con Flask para gestionar modelos y operaciones sobre bases de datos.
- **Flask-CORS:** extensión que permite controlar las solicitudes entre diferentes orígenes, por ejemplo, entre React en el puerto 5173 y Flask en el puerto 5000.
- **CRUD**: operaciones Create, Read, Update y Delete.
- **Variables de entorno:** configuración externa que permite mantener las credenciales de la base de datos fuera del código fuente.

PASOS

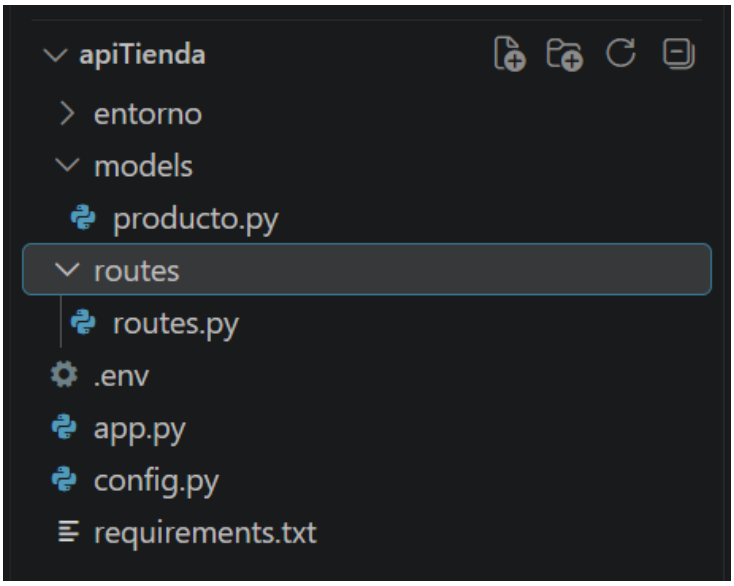
1. Crear la carpeta del proyecto
2. Crear un entorno de desarrollo

python -m venv entorno

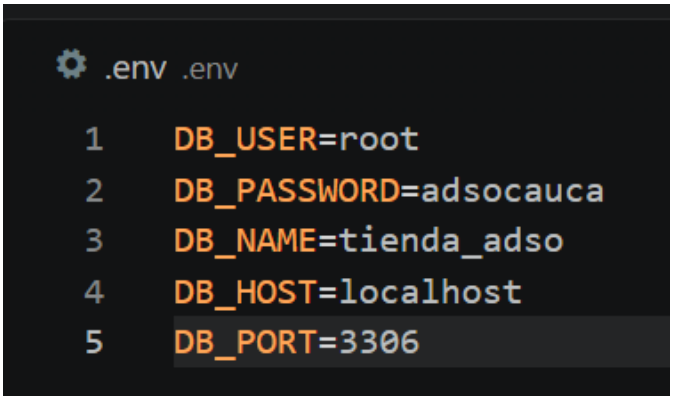
3. **Activar el entorno:** ubicarse en la capeta **entorno/scripts** y ejecutar el archivo **actívate.bat**
4. Instalar las librerías necesarias

pip install Flask Flask-SQLAlchemy Flask-Cors PyMySQL python-dotenv

5. Crear la siguiente estructura de carpetas y archivos



6. Código del archivo **.env**



7. Código del archivo **config.py**

config.py config.py\...

```python
import os
from dotenv import load_dotenv

load_dotenv()

DB_HOST = os.getenv("DB_HOST", "localhost")
DB_PORT = int(os.getenv("DB_PORT", "3306"))
DB_USER = os.getenv("DB_USER", "root")
DB_PASSWORD = os.getenv("DB_PASSWORD", "")
DB_NAME = os.getenv("DB_NAME", "tienda_adso")
```

8. Código del archivo **producto.py** de la carpeta **models**

producto.py models\producto.py\...

```python
from app import db


class Producto(db.Model):
    __tablename__ = "productos"

    idProducto = db.Column( db.Integer,primary_key=True,   autoincrement=True,)
    proCodigo = db.Column(db.String(30),nullable=False,unique=True,)
    proNombre = db.Column(db.String(100), nullable=False,)
    proPrecio = db.Column(db.Float,nullable=False,)
    proCategoria = db.Column(db.String(50),nullable=False,)
    Tabnine | Edit | Test | Explain | Document
    def to_dict(self):
            return {
                "idProducto": self.idProducto,
                "proCodigo": self.proCodigo,
                "proNombre": self.proNombre,
                "proPrecio": self.proPrecio,
                "proCategoria": self.proCategoria
            }

```

Aquí utilizamos el **ORM Flask-SqlAlchemy** donde trabajamos es con objetos, donde para cada tabla se crea una clase.

9. Código del archivo **routes.py** de la carpeta routes

En el arhcivo estamos incluyendo las rutas y también se encarga de validar los datos recibidos, interactuar con SQLAlchemy y generar respuestas HTTP.

routes.py routes\routes.py\...

```python
from flask import request,jsonify
from app import api, db
from models.producto import Producto
from sqlalchemy import exc


Tabnine | Edit | Test | Explain | Document
@api.route("/api/v1/productos/", methods=['POST'])
def crearProducto():
    try:

        datos=request.get_json(force=True)
        """
            producto = Producto(proCodigo=datos['proCodigo'],
                                proNombre=datos['proNombre'],
                                proPrecio=datos['proPrecio'],
                                proCategoria=datos['proCategoria'])
        """
        producto = Producto(**datos)
        db.session.add(producto)
        db.session.commit()
        status=201
        return jsonify({"mensaje":'Producto Agregado correctamente',
                        "Producto": producto.to_dict()}),status
    except exc.SQLAlchemyError as error:
            mensaje=str(error)
            status=400
            return jsonify({"mensaje":mensaje}),status
```



```
Tabnine | Edit | Test | Explain | Document
@api.route("/api/v1/productos/", methods=['GET'])
def obtenerProductos():
    try:
        productos = Producto.query.all()
        listaProductos=[]

        for p in productos:
            listaProductos.append(p.to_dict())
        status=200

        return jsonify({"mensaje":'Productos',
                        "Productos": listaProductos}),status
    except exc.SQLAlchemyError as error:
        mensaje=str(error)
        return jsonify({"mensaje":mensaje}),500

Tabnine | Edit | Test | Explain | Document
@api.route("/api/v1/productos/<int:idProducto>", methods=['GET'])
def obtenerProductoPorID(idProducto):
    try:
        producto = Producto.query.get(idProducto)
        if producto is None:
            mensaje="No existe producto con ese ID"
            status=404
        else:
            producto=producto.to_dict()
            mensaje="Producto encontrado"
            status=200

        return jsonify({"mensaje":mensaje,
                        "Producto": producto}),status
    except exc.SQLAlchemyError as error:
        mensaje=str(error)
        status=500
        return jsonify({"mensaje":mensaje}),status
```

```
Tabnine | Edit | Test | Explain | Document
@api.route("/api/v1/productos/<int:idProducto>", methods=['PUT'])
def actualizarProducto(idProducto):
    try:
        producto = Producto.query.get(idProducto)
        if producto is None:
            mensaje="No existe producto con ese ID"
            status=404
        else:
            datos = request.get_json(force=True)
            producto.proCodigo=datos['proCodigo']
            producto.proNombre = datos['proNombre']
            producto.proPrecio = datos['proPrecio']
            producto.proCategoria = datos['proCategoria']
            db.session.commit()
            status=200

            mensaje="Producto actualizado"
        return jsonify({"mensaje":mensaje,
                                "Producto": producto.to_dict()}),status
    except exc.SQLAlchemyError as error:
        mensaje=str(error)
        status=500
        return jsonify({"mensaje":mensaje}),status

```

```
Tabnine | Edit | Test | Explain | Document
@api.route("/api/v1/productos/<int:idProducto>", methods=['DELETE'])
def eliminarProducto(idProducto):
    try:
        producto = Producto.query.get(idProducto)
        if producto is None:
            mensaje="No existe producto con ese ID"
            status=404
        else:
            db.session.delete(producto)
            db.session.commit()
            mensaje="Producto Eliminado"
            status=200
        return jsonify({"mensaje":mensaje}),status
    except exc.SQLAlchemyError as error:
        mensaje=str(error)
        status=500
        return jsonify({"mensaje":mensaje}),500
```

10. Código del archivo **app.py** quien es el archivo que crea la aplicación **flask** llamada **api**.

app.py app.py\...

```python
from flask import Flask
from flask_sqlalchemy import SQLAlchemy
from flask_cors import CORS
from config import *

#Se crea el ojeto de tipo Flask
api = Flask(__name__)

#Se crea un objeto cors que permite que se pueda acceder desde cualquier sitio
cors = CORS()

#se crea la cadena de conexion a mysql
cadenaConexion=f"mysql+pymysql://{DB_USER}:{DB_PASSWORD}@{DB_HOST}/{DB_NAME}"

api.config['SQLALCHEMY_DATABASE_URI']=cadenaConexion

#se crea un objeto de tipo SqlAlchemy asociado a la aplicación Flask
db = SQLAlchemy(api)

#Importamos Todo lo que tenga el archivo routes.py
from routes.routes import *


if __name__=='__main__':
    with api.app_context():
        #se crean las tablas que se encuentren en el modelo si no existen
        db.create_all()
    #se ejecuta la aplicación flask
    api.run(port=5000,debug=True)
```

11. Exportar las librerías a archivo requirements.txt

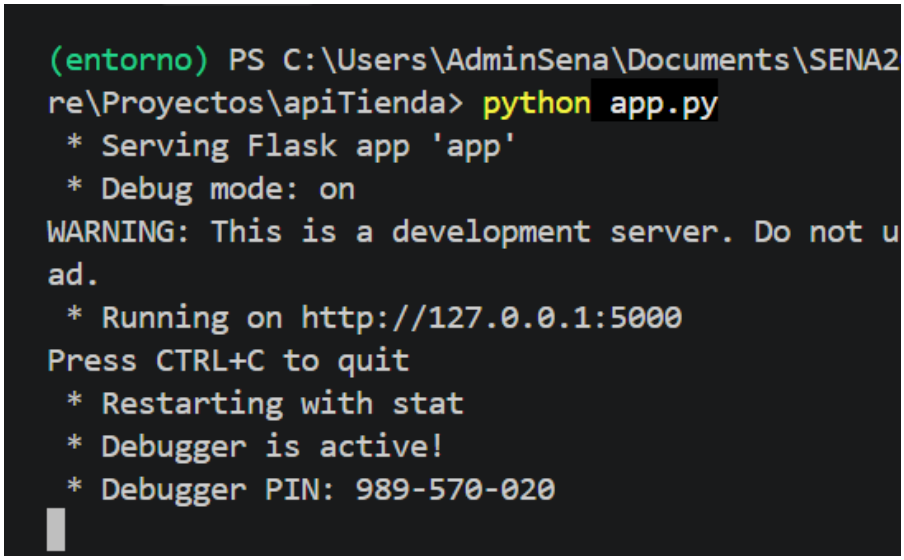
Pip freeze > requirements.txt

El archivo anterior nos sirve para guardar el nombre de las librerías con sus versiones para posteriormente usarse para instalar cuando se requiera

12. Crear archivo **.gitignore** excluyendo el archivo **.env**

13. Ejecutar la aplicación y verificar:

Python app.py



14. Tabla de rutas **CRUD** de la **API**

Todas las rutas de productos comienzan con **/api/v1/productos**

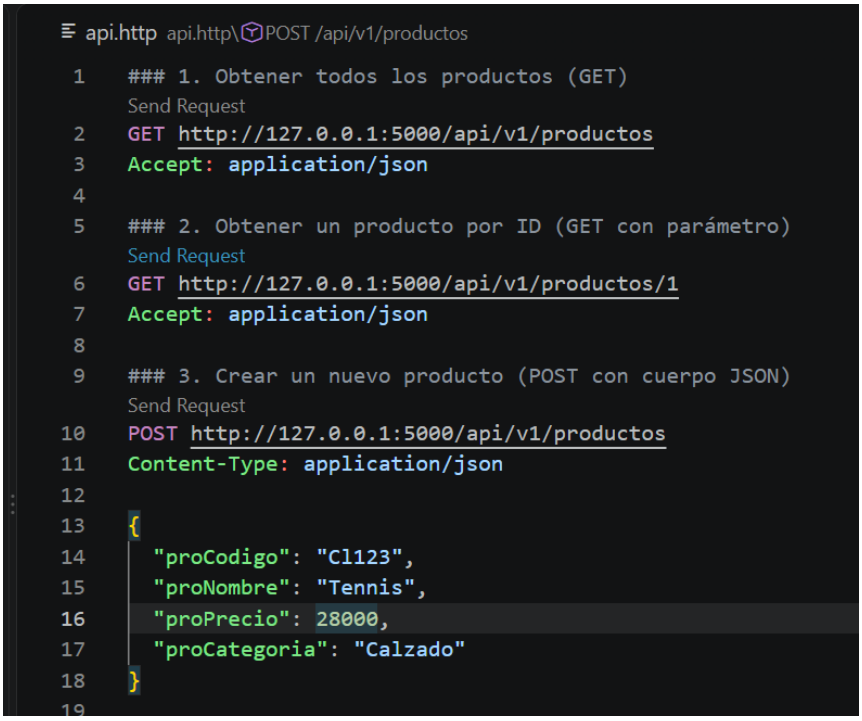
Método	Endpoint	Descripción	Respuesta Exitosa
POST	api/v1/productos	Registra un producto	201
GET	api/v1/productos	Obtener todos los productos	200
GET	api/v1/productos/id	Obtener producto por ID	200
PUT	api/v1/productos/id	Actualizar producto de acuerdo a ID	200
DELETE	api/v1/productos/id	Eliminar el producto de acuerdo a ID	200

Otros códigos de respuesta importantes:

- 400 Bad Request: JSON inválido, campos faltantes o datos incorrectos.
- 404 Not Found: el producto solicitado no existe.
- 409 Conflict: el código del producto está duplicado o se incumple una restricción de integridad.
- 500 Internal Server Error: error inesperado durante una operación con la base de datos.

15. Hacer la prueba desde **postman** o creando un archivo denominado **api.http.** Instalar la extensión en visual studio code llamada **REST CLIENT**

16. Ejemplo cCódigo del archivo **api.http**



```
api.http  api.http\POST /api/v1/productos
### 1. Obtener todos los productos (GET)
Send Request
GET http://127.0.0.1:5000/api/v1/productos
Accept: application/json

### 2. Obtener un producto por ID (GET con parámetro)
Send Request
GET http://127.0.0.1:5000/api/v1/productos/1
Accept: application/json

### 3. Crear un nuevo producto (POST con cuerpo JSON)
Send Request
POST http://127.0.0.1:5000/api/v1/productos
Content-Type: application/json

{
  "proCodigo": "Cl123",
  "proNombre": "Tennis",
  "proPrecio": 28000,
  "proCategoria": "Calzado"
}
```

Se prueban dando clic en la parte superior de cada petición donde dice **Send Request**.

Como ejemplo al dar clic en la primera, la de obtener todos los productos



```
Response(11ms)

HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.13.0
Date: Mon, 26 Oct 2026 16:57:45 GMT
Content-Type: application/json
Content-Length: 349
Connection: close

{
  "Productos": [
    {
      "idProducto": 1,
      "proCategoria": "Calzado",
      "proCodigo": "31",
      "proNombre": "Tennis",
      "proPrecio": 25000.0
    },
    {
      "idProducto": 4,
      "proCategoria": "Calzado",
      "proCodigo": "32",
      "proNombre": "Botas",
      "proPrecio": 32000.0
    }
  ],
  "mensaje": "Productos"
}
```

17. Subir el proyecto a un repositorio personal.

18. Crear archivo .**md** para el proyecto en **Gitub** con la descripción del proyecto.

Actividad complementaria:

Crear una aplicación web con react vite que consuma la api anterior realizando todas las tareas como: agregar producto, obtener todos los productos, consultar producto por su id, actualizar producto y eliminar producto.

Fecha de Entrega: 06/11/2026
Forma de Entrega: Url del repositorio donde quedó la aplicación, y vídeo donde explican su funcionamiento. Donde se evidencie tanto el proyecto en react como la api.

Referencias:

1. Flask documentation. https://flask.palletsprojects.com/en/stable/Referencia para la creación de aplicaciones Flask, manejo de solicitudes HTTP, respuestas JSON, configuración y organización del backend.

2. Quickstart. https://flask-sqlalchemy.palletsprojects.com/en/stable/quickstart/
Consulta para configurar SQLAlchemy en Flask, definir modelos, gestionar sesiones y ejecutar operaciones CRUD.

3. Flask-CORS documentation. https://flask-cors.readthedocs.io/en/latest/
Referencia para configurar el intercambio de recursos entre orígenes distintos y permitir que React consuma la API.

4. SQLAlchemy documentation. https://docs.sqlalchemy.org/en/20/
Documentación para el mapeo objeto-relacional, consultas, restricciones de integridad y transacciones con bases de datos.

5. https://pymysql.readthedocs.io/en/latest/ Referencia para el controlador de conexión entre Python y MySQL utilizado por SQLAlchemy.

6. Python Software Foundation. (s. f.). The Python tutorial. https://docs.python.org/3/tutorial/
Material de consulta para los fundamentos del lenguaje Python, módulos, funciones, excepciones y organización del código.

7. Theskumar. (s. f.). python-dotenv. GitHub. https://github.com/theskumar/python-dotenv

CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	César Marino Cuéllar Chacón	Instructor	CTPI-CAUCA	30-10-2026